

FINAL PROJECT DESIGNED DOCUMENT

TEAM 5

TEAM MEMBERS:

ALBERT NII OKAI QUAYE ACROMOND

KOFI ASANTE GYAMEBI

KINGSLEY WUNZOOYA NAHYI

GERARD LEDOUX MENDJEMO NGANGOM

WILLIEWISE OPPONG



(COHORT C, CS213)

OBJECT-ORIENTED PROGRAMMING

FACULTY: DR. DANIEL ADDO

PROJECT TITLE: SOLAR POTENTIAL CALCULATOR FOR ROOFTOPS

PROJECT DESCRIPTION AND OVERVIEW:

To use solar energy effectively, households or users need to be able to calculate the solar potential accurately. However, most households are unable to do this easily and feasibly because they lack tools. The app aims to fix this problem. It does this by making an estimate of the rooftop solar energy capacity and going on to identify the household appliances that can be powered by it. It is a Java program that makes use of important OOP principles, relating real-world components using classes. To obtain estimates, the program tries to use real-time illuminance data from smartphones, which have been proven to be correlated linearly with solar irradiance (Di Laccio, Ferrero, Muscas and Serpi, 2019). The readings are then transformed into direct normal irradiance and compared to the recommended value of 600 W/m^2 suggested by International Electrotechnical Commission(IEC,2016).. The program combines these values with data about the rooftops and factors in panel efficiency (GreenMatch, 2025) to calculate the solar potential of the rooftop and goes on to provide users with recommendations for appliances they can power using that energy.

PROJECT FUNCTIONALS AND OBJECTIVES:

1. It should be able to retrieve user inputs, in the form of real-time illuminance data from the sensors as well as dimensions from the rooftop. It uses orientation and shading too.,
2. It then converts the illuminance to irradiance to be used in the potential energy calculation.
3. It calculates the potential energy by checking the specific types of solar panels used.
4. It then recommends appliances that can be powered feasibly based on potential energy.

HIGH-LEVEL ARCHITECTURE

INPUT

Our app combines two approaches for collecting input. It accepts real-time sensor data input and parameters gotten from the user through scanner inputs.. Our app uses a wireless sensor app called Phyphox, which captures luminance values in lux. We connect our application to the phyphox app using hotspot, which generates the lux values in JSON format, which is converted to a double by parsing. The system collects user parameters through a console interface. These parameters include rooftop physical characteristics (area, shading factor, orientation), solar panel type selection(monocrystalline or polycrystalline), and optional calibration settings using reference solar irradiance values.

PROCESSING

The first input (phyphox's URL) is passed into the PhyphoxClient's class constructor as an argument after it is trimmed. The PhyphoxClient class opens a connection to the URL, uses the

buffer reader to read it in JSON format, which is then put into a string builder via a while loop. The string builder is then converted to a string and parsed to a double and returned. The next inputs: area of the rooftop, shading of the rooftop, and orientation of the rooftop are processed in the rooftop class. The rooftop class has a method `getEffectiveArea()`, which basically returns a double, calculated by multiplying the three variables of the rooftop class. The next two inputs are used to choose the panel type used in the calculation and the calibration (standard or reference DNI). The calculated variables are then passed into the SolarCalculator class, which checks and displays in real time a list of devices that could be powered with the energy generated.

OUTPUT

The application displays in real time the appliances that could be powered by the energy generated based on the amount of light the phone's sensor receives. If an appliance could be powered, it is marked with a check box, and if not, it is marked with an unchecked box.

Class Descriptions & Responsibilities

The final implementation relies on the following interactions:

Core Logic

- **PhyphoxSolarMonitor (Main Class):**
 - **Responsibility:** Acts as the controller. It handles user inputs (configuration), initiates the network connection, and runs the main monitoring loop. It includes robust error handling (exponential backoff) to manage network instability.
 - **Key Feature:** Implements a dynamic "Calibration Mode" allowing users to choose

between a standard luminous efficacy factor or a reference-based calibration.

- **SolarCalculator:**

- **Responsibility:** The computational engine. It aggregates data from the Rooftop, SolarPanel, and Calibration classes to calculate the final power output.
- **Logic:** It applies the formula: $P_{out} = \text{Irradiance} \times \text{PanelEfficiency} \times \text{EffectiveArea} \times \text{LossFactors}$.

- **Calibration:**

- **Responsibility:** Manages the conversion of Illuminance (Lux) to Direct Normal Irradiance (DNI).
- **Logic:** Defaults to a standard factor (0.0079 W/m^2 per Lux) but can be updated at runtime if the user provides a known reference value, following the methodology of Di Laccio et al. (2019).

Networking

- **PhyphoxClient:**

- **Responsibility:** Encapsulates all networking logic. It connects to the Phyphox REST API, retrieves the JSON stream, and parses it using regex to extract the most recent illuminance value.
- **Design Decision:** Kept separate from the calculation logic to allow for easy replacement if the sensor source changes (e.g., switching to a different sensor app).

Models

- **Rooftop:**

- **Responsibility:** Represents the installation site. It calculates the "Effective Area" by

penalizing the raw area based on shading (0.0–1.0) and orientation (South, North, East, West).

- **Logic:** Uses orientation factors (e.g., South = 1.0, North = 0.6) to simulate real-world efficiency losses.
- **SolarPanel (Abstract Class):**
 - **Responsibility:** Defines the contract for all photovoltaic panels, ensuring they possess efficiency and area attributes.
 - **Polymorphism:** Allows the system to treat different panel types uniformly.
- **MonocrystallinePanel (Concrete Class):**
 - **Context:** Extends SolarPanel. Represents higher-efficiency, more expensive panels (GreenMatch, 2025).
 - **Attributes:** Includes a temperatureCoeff to model performance loss in heat.
- **PolycrystallinePanel (Concrete Class):**
 - **Context:** Extends SolarPanel. Represents lower-efficiency, budget-friendly panels.
 - **Attributes:** Includes a degradationFactor to model faster wear or lower light sensitivity.
- **Appliance & ApplianceDatabase:**
 - **Responsibility:** Appliance is a simple data object (name, wattage). ApplianceDatabase acts as a repository, providing a predefined list of household devices for the feasibility report.

Key Design Decisions & Justifications

- **Use of Polymorphism for Solar Panels**
 - **Decision:** We created an abstract SolarPanel class with concrete subclasses (MonocrystallinePanel, PolycrystallinePanel).

- **Justification:** This allows for extensibility. If we want to add "Thin-Film" panels in the future, we simply create a new subclass without modifying the core SolarCalculator logic. It also reflects the real-world trade-offs discussed by GreenMatch (2025), where different panels have different physical properties (efficiency vs. degradation).
- **Separation of Calibration Logic**
 - **Decision:** Calibration logic was extracted into its own class rather than being hardcoded in the main loop.
 - **Justification:** Converting Lux to W/m² is scientifically complex. By isolating this, we can swap the mathematical model (e.g., adding a linear regression model) without breaking the rest of the application. This aligns with the Single Responsibility Principle.
- **Effective Area Calculation in Rooftop**
 - **Decision:** The Rooftop class exposes a getEffectiveArea() method rather than just raw dimensions.
 - **Justification:** A 100m² roof facing North produces significantly less power than one facing South. Encapsulating this logic within the Rooftop class ensures that the SolarCalculator receives data that already accounts for environmental constraints, simplifying the final power formula.
- **Using Phyphox over regular input values:** Phyphox allowed us to collect luminance from real light sources which provides a real world experience in our project compared to calculations made using regular input values

- **Using an abstract class for Solar Panel**
 - We used an abstract Solar Panel to have a main template or class from which other panels can follow. It was also abstract so that the other classes would be able to calculate based on specifications. Why separation of calibration
 - Separating the calibration from the main solar power algorithm helped to make our code modular and reusable.
- **Using try catch blocks:** We used try catch blocks for error-handling and ensure robustness of our code. These exceptions helps to safeguard against unexpected failures that may occur when collecting input from the user or during network communications with Phyphox servers.
- **Why we used an appliance database:** The appliance database uses an array list that gives a predefined catalog of appliances and their wattages. Since these list are not static but mutable, appliances can be added and removed which made it an excellent choice.

Testing and Error Handling

To ensure the reliability of the system, we implemented a rigorous testing strategy and robust error handling mechanisms.

Unit & Integration Testing

We used the JUnit 5 framework to validate business logic before deployment. Key tests include:

- **Rooftop Calculations:** Validating that orientation factors (e.g., North vs. South) correctly reduce the effective area.

- **Panel Polymorphism:** Ensuring that **MonocrystallinePanel** and **PolycrystallinePanel** compute outputs differently based on their specific efficiency and loss coefficients.
- **Integration Testing:** We implemented a **testSolarCalculatorIntegration** test to verify the interaction between the **Rooftop**, **SolarPanel**, and **SolarCalculator** classes. This critical test helped identify and resolve a "double-counting" logic error where the rooftop area was being multiplied twice in the calculation chain.

5.2. Error Handling Mechanisms

- **Exponential Backoff for Connectivity:** The system interacts with a smartphone over Wi-Fi, which can be unstable. In the **PhyphoxSolarMonitor** main loop, we implemented an exponential backoff strategy. If the connection fails, the system waits for increasing intervals (1s, 2s, 4s...) before retrying, preventing the application from crashing due to transient network drops.
- **Input Validation:** User inputs (e.g., rooftop area, shading) are wrapped in validation loops (**getValidDouble**). This prevents the application from terminating unexpectedly if a user inputs non-numeric characters.
- **Graceful Degradation:** If the sensor returns **null** or malformed data, the application logs a warning rather than crashing, maintaining the stability of the monitoring session.

References

- Di Laccio, J. L., Ferrero, A., Muscas, C., & Serpi, A. (2019). *Using smartphone ambient light sensors for solar irradiance measurements: A feasibility study*. *Measurement*, 136, 453–460. <https://arxiv.org/abs/2308.16158>
- GreenMatch. (2025, October 14). *Comparing monocrystalline vs polycrystalline solar panels*. <https://www.greenmatch.co.uk/blog/monocrystalline-vs-polycrystalline-solar-panels>
- International Electrotechnical Commission. (2016). Photovoltaic system performance – Part 3: Energy evaluation method (IEC TS 61724-3:2016). <https://webstore.iec.ch/en/publication/25466>